

Security Measures & Testing

Authentication, Authorization, and Data Protection

Our system uses JWT-based authentication for three types of users: patients, therapists, and admins. After a successful login, the backend generates a token containing the user ID and user type. This token is stored in an HTTP-only cookie and is also returned to the frontend for authenticated API requests. For protected routes, the backend verifies the token before allowing access. If the token is missing, invalid, expired, or belongs to a deactivated account, the request is rejected.

Authorization is handled at more than one level. First, protected routes require a valid token. Second, sensitive actions are checked again in the controller or service layer to confirm that the user has the correct role. For example, admin-only features such as approving therapists, creating admin accounts, viewing admin bookings, and changing booking status are restricted to admin or super-admin users. This layered approach helps reduce the risk of unauthorized access.

To protect sensitive data, user passwords are not stored in plain text. Instead, the system hashes passwords with bcrypt before saving them to the database. In addition, password fields are excluded from normal query results, so they are not accidentally returned in API responses. The password reset process also uses a dedicated reset-token validation flow, so reset tokens cannot be used as normal login tokens.

The project also includes extra protection for the initial super-admin setup process. This setup route can be turned on or off through environment variables, and it requires a separate setup key sent in the request header. This reduces the risk of unauthorized system initialization. In addition, admin login is protected with both request rate limiting and account lockout logic after repeated failed login attempts.

Security Best Practices Followed

Several standard security practices are already used in this project.

First, password security is handled properly. Passwords are hashed with bcrypt and salted before storage. This means that even if the database is exposed, the original passwords are not directly visible.

Second, token handling follows safer defaults. JWT cookies are set as HTTP-only, which helps prevent JavaScript from reading them directly. The cookie settings also use same-site protection

and use secure cookies outside development mode, which helps reduce some risks related to token theft and cross-site request abuse.

Third, the backend has controlled cross-origin access. CORS is not fully open; instead, it is configured with an allowed origin. The backend also supports proxy-aware HTTPS deployment through the TRUST_PROXY setting, which is useful when deploying behind ngrok or a reverse proxy.

Fourth, the application already includes some important input validation. Password strength is checked for admin account creation and super-admin setup. Appointment creation validates required fields such as therapist, date, time, and home-care address when needed. Admin content management also checks required fields and validates numeric values such as ordering. These checks help prevent invalid or incomplete data from entering the system. On the frontend, the project benefits from React's default escaping behavior. During code review, no use of dangerouslySetInnerHTML, eval, or similar unsafe rendering methods was found. This is important because it lowers the risk of users injecting malicious scripts into the interface through normal text fields.

Results of Basic Security Tests

Results of Basic Security Tests

We added basic security-focused tests covering injection prevention and cross-site scripting (XSS) prevention.

For injection prevention, we tested malicious login payloads that attempted to pass object-based values instead of normal credential strings. These requests were rejected before database lookup, showing that the login entry points do not accept injection-style credential formats. Although the application uses MongoDB rather than SQL, this basic test demonstrates protection against common query-injection patterns at the input-validation layer.

For XSS prevention, we tested content containing potentially unsafe strings such as <script> tags and malicious HTML attributes. The frontend rendered these values as plain text rather than executable code, and no unsafe DOM elements were created during rendering. This confirms that the current UI flow benefits from React's default escaping behavior and provides basic protection against stored or reflected XSS in normal text rendering.